

Python 시험 공부 정리본

Lecture 1-4 + 수업 실습 필기 7개 통합

사용 방법

1. 각 파트의 개념을 읽고 코드 출력을 실행 전에 예상한다.
2. 파트 마지막 4문제를 답을 보지 않고 푼다. 총 40문제이다.
3. 막히면 힌트만 보고 다시 푼 뒤 별도 정답·해설과 비교한다.
4. 틀린 문제는 코드의 변수값을 한 줄씩 적고 다음 날 다시 풀어본다.

범위와 기준

첨부한 Lecture 1-4 전체를 학습 범위로 가정했다. 공식 시험 범위나 출제 빈도를 확인한 것은 아니다. 강의자료의 주제·실습 코드를 중심으로 재구성하며, 독립 행 생성·이름 변환 등 보충은 함정 이해를 위한 설명이다. 이미지 속 원문 코드가 텍스트에 온전히 없는 부분은 필기와 Python 문법을 바탕으로 만든 학습용 예제이며 원본의 그대로 전사가 아니다.

정답은 별도 파일로 분리했다. 출력 예측은 각 문제를 독립적으로 실행한다고 가정한다. 아래 코드 블록은 예제 단위로 실행하고 서로 다른 파트의 동일 클래스명은 혼용하지 않는다.

학습 지도

01 변수·입출력·연산자 → 02 조건·반복 → 03 함수 → 04 리스트·2차원·컴프리헨션 → 05 딕셔너리·세트 → 06 문자열·튜플 → 07 클래스 기본 → 08 정보 은닉 → 09 참조·클래스 변수 → 10 특수 메소드

수업 자료 연결

Lecture 1: 파트 01-06 / Lecture1-1.py: 01-03 / Lecture1-2.py: 01,04-06

Lecture 2: 파트 03,07 / Lecture2-1.py: 07

Lecture 3: 파트 07-09 / Lecture3-1.py: 07-08 / Lecture3-2.py: 08-09

Lecture 4: 파트 10 / Lecture4-1.py: 10

PART 01 변수·입출력·연산자

자료: Lecture 1 p.2-11 / Lecture1-1.py, Lecture1-2.py

핵심 개념

변수는 객체에 붙이는 이름이다. `a = 10`은 오른쪽 값을 왼쪽 이름에 연결한다. 대입 `=`와 비교 `==`를 구별한다. Python은 C처럼 변수형을 미리 선언하지 않는다.

- bool: True/False / int: 정수 / float: 실수 / str: 문자열. `type(x)`로 확인한다.
- `input()`의 반환값은 항상 문자열이다. 계산하려면 `int(input())` 또는 `float(input())`로 변환한다.
- `int("3.5")`는 `ValueError`. `float("3.5")`는 가능하며 `int(3.5)`는 3이다.
- `+`, `-`, `*`, `/`, `//`, `%`, `**`: 덧셈, 뺄셈, 곱셈, 나눗셈, 바닥 나눗셈, 나머지, 거듭제곱.
- `/`는 정수끼리 계산해도 float. `//`는 단순 소수점 제거가 아니라 음의 무한대 방향으로 내림한다. `-7 // 3`은 `-3`.
- `+=`는 누적 대입. `<`, `<=`, `>`, `>=`, `==`, `!=`는 비교. `and/or/not`으로 조건을 결합한다.

출력과 서식

```
a, b = 1, 2
print("{}더하기 {}은 {}입니다.".format(a, b, a+b))
print("%d + %d = %d" % (a, b, a+b))
print("%.2f" % 3.14159)
print("A", end=" ")
print("B")
```

첫 두 줄은 각각 1더하기 2은 3입니다. / `1 + 2 = 3`을 출력한다. `%.2f`는 3.14, 마지막 두 줄은 A B를 한 줄에 출력한다. `%d`는 정수, `%f`는 실수, `%s`는 문자열, `%g`는 간결한 숫자 표시이다. 서식과 전달 값의 개수를 맞춘다. `print`는 기본적으로 인수 사이에 공백, 끝에 줄바꿈을 붙인다.

암기 체크

□ 문자열 "100"과 정수 100은 다르다. □ 숫자 문자열 `+`는 연결이다. □ `range`와 슬라이싱 끝값은 제외된다(다음 파트).

마무리 연습문제

문제 01-1

출력 예측: `print(7 / 2, 7 // 2, 7 % 2, 2 ** 3)`

풀이 메모: _____

문제 01-2

`a = input()`에 12, `b = input()`에 3을 입력했다. `print(a+b)`의 결과와 합계 15를 출력하는 수정 코드를 쓰라.

풀이 메모: _____

문제 01-3

오류 수정: `print("%d %d" % 10)`. 오류 원인과 수정 예를 쓰라.

풀이 메모: _____

문제 01-4

직접 작성: 정수 두 개를 입력받아 합과 나머지를 출력하라. 두 번째 수는 0이 아니라고 가정한다.

풀이 메모: _____

막힐 때만 보는 힌트

- 01-1: 연산자별 반환형을 나눠 생각하자.
- 01-2: input 결과의 자료형부터 확인하자.
- 01-3: %d 개수를 세자.
- 01-4: 입력 시 변환하자.

PART 02 조건문·반복문

자료: Lecture 1 p.12-17 / Lecture1-1.py

실행 흐름

if → elif → else 순서로 검사하며 처음 참인 분기 하나만 실행한다. 독립된 if 여러 개와 다르다. 콜론과 들여쓰기를 반드시 맞춘다.

```
a = 100
if a < 100:
    print("작음")
elif a == 100:
    print("같음")
else:
    print("큼")
```

결과는 같음. 조건식의 ==는 대입이므로 ==로 비교한다.

for와 range

range(start, stop, step)은 stop을 포함하지 않는다. range(3)은 0,1,2. range(1,6)은 1~5. range(5,0,-1)은 5,4,3,2,1이다. range 객체 자체가 리스트는 아니지만 반복되는 값은 해당 리스트와 같다.

```
hap = 0
for i in range(1, 6):
    hap += i
print(hap) # 15
```

추적할 때 i와 hap를 표로 적는다: (1,1), (2,3), (3,6), (4,10), (5,15).

while·break·continue

while은 매번 조건을 검사한다. break는 가장 안쪽 반복문을 종료한다. continue는 현재 반복의 나머지를 건너뛰고 다음 반복으로 간다. while의 갱신을 continue 아래에 두면 무한 반복이 될 수 있다.

필기 while True 예제는 print(123) → b += 1 → b == 10 검사 순서라서 123이 10번 출력된다. 중첩 반복은 바깥 3회 × 안쪽 4회 = 안쪽 본문 12회이다.

암기 체크

□ 종료값 제외 □ 누적 변수는 반복문 전에 초기화 □ break와 continue를 구별

마무리 연습문제

문제 02-1

출력 예측: for i in range(2, 8, 2): print(i) (줄별로 쓰라).

풀이 메모: _____

문제 02-2

출력 예측: s=0; 1~5 반복에서 i==3이면 continue, 나머지는 s+=i. 마지막 s는?

풀이 메모: _____

문제 02-3

직접 작성: 1~20 중 3의 배수만 출력하라.

풀이 메모: _____

문제 02-4

오류 수정: i=0인 while i<3 본문에서 i==1이면 continue를 먼저 실행하고 그 아래에서 i+=1한다. 왜 끝나지 않으며 어떻게 수정하는가?

풀이 메모: _____

막힐 때만 보는 힌트

- 02-1: stop은 제외된다.
- 02-2: 3만 빼고 더한다.
- 02-3: 나머지가 0인지 검사한다.
- 02-4: continue 후 어디로 돌아가는가?

PART 03 함수·인수·변수 범위

자료: Lecture 1 p.18-23, Lecture 2 p.15-16 / Lecture1-1.py

함수 기본

```
def plus(v1, v2):  
    return v1 + v2  
hap = plus(100, 200)  
print(hap) # 300
```

v1/v2는 매개변수, 100/200은 인수. return은 값을 호출 위치로 돌려주고 함수를 종료한다. print는 화면 출력이며 반환과 다르다. return이 없으면 None을 반환한다.

지역 변수·전역 변수

함수 안에서 a = 10으로 대입하면 보통 지역 변수이다. 바깥 a = 20과 별개이다. 함수 안에서 지역 a 없이 읽기만 하면 바깥 a를 읽을 수 있다. 함수 안에서 global a 선언 후 대입하면 전역 a를 변경한다.

필기 첫 func1/func2는 10,20을 출력한다. global을 넣은 두 번째 예제는 10,10을 출력한다. 함수 정의가 다시 나오면 같은 이름이 새 함수에 연결되므로 예제는 구간별로 실행해서 비교하자.

기본값·위치·키워드 인수

```
def greet(name, msg="별일없죠?):  
    return name + ", " + msg  
print(greet("영희"))  
print(greet(msg="잘 지내?", name="철수"))
```

일반 호출에서는 위치 인수를 먼저, 키워드 인수를 뒤에 둔다. f(x=10, 20)은 SyntaxError. 같은 매개변수에 값을 중복 전달하면 TypeError.

여러 결과

필기 multi(100,200)는 [300,-100]이라는 리스트 하나를 반환한다. 두 결과를 리스트에 담은 것이다. return v1+v2, v1-v2는 튜플 하나를 반환한다. 둘 다 hap, sub = multi(...)처럼 언패킹할 수 있다(원소 수와 변수 수가 같아야 한다).

암기 체크

□ 출력과 반환 구분 □ 기본값 □ global의 역할 □ 여러 반환값의 실제 자료형

마무리 연습문제

문제 03-1

출력 예측: `def f(x): print(x+1)` 다음에 `a=f(2); print(a)`를 실행한다.

풀이 메모: _____

문제 03-2

전역 `x=5`. `def f():` 안에서 `x=9`를 대입한 뒤 `f()` 호출 후 `print(x)`를 실행하면?

풀이 메모: _____

문제 03-3

`def sub(x,y,z=0): return x-y-z` 일 때 `sub(10,y=3)`과 `sub(z=2,y=3,x=10)`의 값은?

풀이 메모: _____

문제 03-4

직접 작성: 두 정수를 받아 합과 차를 리스트로 반환하는 `multi` 함수를 만들고 8,3의 두 결과를 출력하라.

풀이 메모: _____

막힐 때만 보는 힌트

- 03-1: `return`이 있는가?
- 03-2: 함수 안 대입에 `global`이 있는가?
- 03-3: 기본 `z`는 0이다.
- 03-4: 리스트 두 원소를 반환하자.

PART 04 리스트·2차원 리스트·컴프리헨션

자료: Lecture 1 p.27-37 / Lecture1-2.py

리스트 기본

리스트는 순서가 있고 원소를 바꿀 수 있는 가변 자료형이다. 인덱스는 0부터, -1은 마지막. `a[start:stop]`은 stop 직 전까지의 새 리스트이다.

```
a = [10, 20, 30, 40]
a[1] = 200
a.append(0)
print(a)          # [10, 200, 30, 40, 0]
print(a[1:3])     # [200, 30]
```

`append(x)`는 맨 뒤에 한 원소 추가, `insert(i,x)`는 삽입, `extend(iterable)`은 여러 원소 추가. `pop(i)`는 삭제한 원소 반환, `remove(x)`는 첫 일치값 삭제. `index(x)`는 위치, `count(x)`는 개수. `sort()/reverse()`는 원본을 바꾸며 반환값은 `None`이다. 없는 값을 `remove/index`로 찾으려 하면 `ValueError`. 잘못된 단일 인덱스는 `IndexError`.

2차원 리스트

```
rows = []
value = 1
for i in range(3):
    row = []
    for k in range(4):
        row.append(value)
        value += 1
    rows.append(row)
```

결과는 `[[1,2,3,4],[5,6,7,8],[9,10,11,12]]`. `rows[1][2]`는 7이다. 필기는 `list1`을 바깥 반복마다 새 리스트로 바뀌 같은 효과를 낸다. `[[111],[222],...,[999]]`는 3×3이 아니라 9×1이다.

컴프리헨션

```
squares = [n*n for n in range(1, 6)]
multiples = [n for n in range(1, 21) if n % 3 == 0]
grid = [[0 for _ in range(4)] for _ in range(3)]
```

각각 `[1,4,9,16,25]`, `[3,6,9,12,15,18]`, 독립된 3개 행의 3×4 리스트이다. `_`는 사용하지 않는 반복 변수의 관례적 이름이다.

보충: 같은 행 공유 함정

`[[0]*4]*3`은 같은 안쪽 리스트를 3번 참조한다. 한 행의 원소를 바꾸면 모든 행에서 바뀐다. 독립된 행은 `[[0]*4 for _ in range(3)]`로 만든다.

암기 체크

□ [행][열] □ 슬라이스 끝값 제외 □ append와 extend □ 행을 새로 생성

마무리 연습문제

문제 04-1

`a=[10,20,30,40]`일 때 `a[-1]`, `a[1:3]`, `a[:2]`를 쓰라.

풀이 메모: _____

문제 04-2

`a=[1,2]`; `a.append([3,4])` 이후 `a`의 값과 길이를 쓰라.

풀이 메모: _____

문제 04-3

직접 작성: 1~10 중 짝수의 제곱만 담는 컴프리헨션을 쓰라.

풀이 메모: _____

문제 04-4

`g=[[0]*2]*2`; `g[0][0]=9` 이후 `g`는? 독립된 행으로 고치는 코드도 쓰라.

풀이 메모: _____

막힐 때만 보는 힌트

- 04-1: 끝 인덱스를 제외한다.
- 04-2: `append`는 한 원소 추가이다.
- 04-3: 수식과 `if` 조건을 분리하자.
- 04-4: 안쪽 리스트가 몇 개 생성되는가?

PART 05 딕셔너리·세트

자료: Lecture 1 p.38-45 / Lecture1-2.py

딕셔너리

{키: 값}으로 생성한다. 필기의 주석에 있는 [키:값]은 잘못된 표기이며 실제 코드처럼 중괄호를 쓴다. 키는 유일하고 같은 키를 반복하면 마지막 값이 남는다.

```
student = {"학번": 2600025, "이름": "김도경", "학과": "컴퓨터학과"}
student["연락처"] = "010-1111-2222"
student["학과"] = "파이썬학과"
del student["학과"]
```

새 키면 추가, 기존 키면 수정, del은 항목 삭제이다. 키는 해시 가능한 값이어야 한다(문자열/정수 가능, 리스트 불가).

- d[key]: 없으면 KeyError. d.get(key): 없으면 None. d.get(key, 기본값): 없으면 기본값.
- keys(), values(), items(): 각각 키/값/키·값 쌍의 뷰(view). 리스트가 필요하면 list(...)로 변환한다.
- key in d는 값이 아니라 키 존재 여부를 검사한다.

```
for k, v in student.items():
    print(k, v)
```

자료·필기 바로잡기

현대 Python(3.7 이상)의 dict는 삽입 순서를 보장한다. 강의의 “순서 보장 없음”은 구버전 설명이다. 기존 키 값 변경은 순서를 바꾸지 않는다. get은 “아무것도 반환하지 않음”이 아니라 None을 반환한다. 필기에서 학번 2000으로 재생성한 student1의 학과는 컴퓨터학과이며 values는 dict_values([2000, '김도경', '컴퓨터학과'])이다. 값 2000은 문자열이 아닌 정수이다. .items()만 적으면 스크립트에서 출력되지 않으므로 print(...)를 사용한다.

세트

중복 없는 원소의 집합. 순서와 인덱싱을 보장하지 않는다. 빈 세트는 set(), {}는 빈 딕셔너리이다. 세트는 독립된 자료형이며 “키만 가진 dict”는 개념적 비유이다.

A={1,2,3,4,5}, B={4,5,6,7}일 때:

- A & B / intersection: {4,5}
- A | B / union: {1,2,3,4,5,6,7}
- A - B / difference: {1,2,3}
- A ^ B / symmetric_difference: {1,2,3,6,7}

원소를 나열한 순서는 정답 판정 대상이 아니다.

암기 체크

☐ 키/값 구별 ☐ get과 [] ☐ view와 list ☐ {}와 set()

마무리 연습문제

문제 05-1

`d={"a":1,"a":9,"b":2}`. `d["a"]`, `list(d.values())`, `"a" in d`의 결과는?

풀이 메모: _____

문제 05-2

`d={"x":10}`. `d.get("y")`, `d.get("y",0)`, `d["y"]`의 차이는?

풀이 메모: _____

문제 05-3

`A={1,2,3}`, `B={3,4}`. 교집합·합집합·차집합 `A-B`·대칭 차집합을 쓰라.

풀이 메모: _____

문제 05-4

직접 작성: `scores={"민수":80,"영희":95}`의 이름과 점수를 `items()`로 한 줄씩 출력하라.

풀이 메모: _____

막힐 때만 보는 힌트

- 05-1: 같은 키의 마지막 값이다.
- 05-2: 없는 키 처리만 비교한다.
- 05-3: 공통·전체·한쪽·공통 제외.
- 05-4: 키와 값을 두 변수로 받자.

PART 06 문자열·map·튜플

자료: Lecture 1 p.46-54 / Lecture1-2.py

문자열은 불변

인덱싱과 슬라이싱은 리스트처럼 가능하지만 `s[0] = 'A'`로 원본 원소를 바꿀 수 없다(`TypeError`). 새 문자열을 만들어 변수에 다시 대입해야 한다. `+`는 연결, `*`는 반복, `len`은 문자 수이다.

```
s = "자료구조&알고리즘"
print(s[0], s[1:4], s[4:]) # 자 료구조 &알고리즘
```

- `count`: 부분 문자열 개수. `find`: 첫 위치, 없으면 -1. `rfind`: 마지막 위치를 원래 문자열의 왼쪽 기준 인덱스로 반환.
- `index`: 첫 위치, 없으면 `ValueError`.

- `split()`: 공백 기준으로 나누어 리스트 반환. `split(',')`처럼 구분자 지정 가능.

- '구분자'.`join`(문자열들의 반복 가능한 객체): 문자열들을 연결한다. 원소는 문자열이어야 한다.

```
before = ['2022', '12', '31']
after = list(map(int, before))
print(after) # [2022, 12, 31]
print('-'.join(before)) # 2022-12-31
```

`map`은 함수를 각 원소에 적용하는 이터레이터를 반환한다. `list`를 씌워 결과를 리스트로 만든다.

튜플

튜플은 원소의 연결을 바꿀 수 없는 불변 자료형. `t=(10,20,30)`, `t=10,20,30` 모두 튜플이다. `(10)`은 `int`, `(10,)`는 튜플이다. 하나일 때 심표가 핵심이다. 읽기·슬라이싱·연결·반복은 가능하고 원소 대입/삭제와 `append`는 불가하다.

```
t = (10, 20, 30)
a = list(t)
a.append(40)
t = tuple(a)
print(t) # (10, 20, 30, 40)
```

기존 튜플을 수정한 것이 아니라 새 튜플을 만들어 이름을 재연결했다. `del t`는 이름을 삭제하는 문장이지 함수가 아니다. 튜플 안에 리스트가 있다면 그 리스트 내부는 바뀔 수 있다(보충).

암기 체크

☐ `find` -1 / `index` 예외 ☐ `join` 호출 방향 ☐ `map` 결과 ☐ 한 원소 튜플 심표

마무리 연습문제

문제 06-1

`s="ababa". s.count("a"), s.find("ba"), s.rfind("ba"), s.find("z")`는?

풀이 메모: _____

문제 06-2

"10,20,30"을 정수 리스트로 바꾸는 한 줄 코드를 쓰라.

풀이 메모: _____

문제 06-3

`a=(10); b=(10,)`. 두 자료형과 `b[0]=20`의 결과는?

풀이 메모: _____

문제 06-4

직접 작성: ["Python","시험","공부"]를 /로 연결한 문자열을 출력하라.

풀이 메모: _____

막힐 때만 보는 힌트

- 06-1: 문자 인덱스를 0부터 적자.
- 06-2: 분리 → 변환 → 리스트화.
- 06-3: 튜플을 만드는 핵심은 쉼표.
- 06-4: 구분자 문자열이 메소드의 주체.

PART 07 객체지향·클래스·생성자·self

자료: Lecture 2 p.2-20, Lecture 3 p.2-3 / Lecture2-1.py, Lecture3-1.py

개념 연결

절차지향은 작업을 함수·절차 중심으로 구성한다. 객체지향은 관련 데이터(속성)와 동작(메소드)을 객체에 묶는다. 클래스는 설계도, 인스턴스는 그 클래스로 만든 개별 객체이다. 캡슐화는 데이터·동작을 묶고 공용 인터페이스 뒤로 구현을 감추는 것이다.

Counter 단계별 이해

```
class Counter:
    def __init__(self, initValue=0):
        self.count = initValue
    def increment(self):
        self.count += 1

a = Counter(100)
b = Counter()
a.increment()
print(a.count, b.count) # 101 0
```

① Counter(100)으로 객체 생성 ② __init__의 self가 새 객체를 가리킴 ③ self.count에 100 저장 ④ a에 객체 연결 ⑤ a.increment()에서 self는 a가 가리키는 객체. a와 b의 count는 독립적이다.

수업에서는 __init__을 생성자라 부른다. 엄밀히는 생성된 객체를 초기화하는 메소드이다. 보통 인스턴스 변수를 정의·초기화한다. __init__은 값을 반환하지 않는다.

self는 예약어가 아닌 관례적 이름이며 인스턴스 메소드의 첫 매개변수이다. a.increment() 호출에서는 자동 전달되므로 직접 인수로 쓰지 않는다. self.count(객체 속성)과 count(지역 이름)는 다르다.

TV·Car 필기 연결

TV는 channel/volume/on을 속성으로 갖는다. setChannel은 self.channel을 바꾸고 getChannel은 값을 반환한다. Television(9,10,True) → show → setChannel(11) → show는 9 10 True / 11 10 True이다.

Car.drive()는 self.speed=60을 실행한다. Car(0,'blue','E-class')에서 drive 호출 전 속도 0, 이후 60.

Lecture2-1.py의 volume은 오타지만 매개변수와 대입 우변이 일치해서 위치 인수 호출은 동작한다. volume 키워드를 쓰려면 매개변수명도 volume으로 통일해야 한다.

암기 체크

☐ 클래스/객체/인스턴스 ☐ 속성/메소드 ☐ self ☐ 초기화 기본값

마무리 연습문제

문제 07-1

위 Counter 정의에서 `a=Counter(3); b=Counter(); a.increment(); a.increment(); b.increment()` 후 `a.count`, `b.count`는?

풀이 메모: _____

문제 07-2

오류 수정: `def increment(self): count += 1`. 객체 속성을 증가시키도록 고쳐라.

풀이 메모: _____

문제 07-3

클래스와 인스턴스의 차이를 TV 예시로 두 문장 안에 설명하라.

풀이 메모: _____

문제 07-4

직접 작성: `channel`을 초기화하고 `setChannel/getChannel`을 제공하는 `Television` 클래스를 작성하라. 초기 채널 9를 11로 바꾸고 출력하라.

풀이 메모: _____

막힐 때만 보는 힌트

- 07-1: `a`와 `b`를 따로 추적한다.
- 07-2: 지역 이름 아닌 객체 속성.
- 07-3: 설계도와 제품의 관계.
- 07-4: setter는 변경, getter는 반환.

PART 08 정보 은닉·접근자·은행 계좌

자료: Lecture 3 p.4-9 / Lecture3-1.py, Lecture3-2.py

__이름과 이름 변환

self.__age처럼 앞에 밑줄 두 개를 붙이면 클래스 이름을 포함한 이름으로 변환(name mangling)된다. Student의 __age는 _Student__age 형태가 된다. obj.__age로 직접 읽으면 AttributeError. 완전한 보안 장벽은 아니므로 “외부에서 절대로 접근 불가”라고 이해하지 않는다.

```
class Student:
    def __init__(self, name=None, age=0):
        self.__name = name
        self.__age = age
    def getAge(self):
        return self.__age
    def setAge(self, age):
        self.__age = age
obj = Student('Hong', 20)
obj.setAge(21)
print(obj.getAge()) # 21
```

getter는 값을 읽어 반환, setter는 값을 설정. 접근 경로를 메소드로 모으면 검증을 추가할 수 있다. 위 수업 코드 자체에는 나이 검증이 없으므로 음수도 대입된다. 검증을 추가하는 것은 확장이지 원본 동작이 아니다.

BankAccount

```
class BankAccount:
    def __init__(self):
        self.__balance = 0
    def deposit(self, amount):
        self.__balance += amount
        return self.__balance
    def withdraw(self, amount):
        self.__balance -= amount
        return self.__balance
```

a.deposit(100)은 잔액 100 반환, a.withdraw(10)은 90 반환. 필기의 메시지는 “통장에 출금 / 통장에서 입금”으로 표현이 뒤바뀌어 있으나 계산식 += 입금, -= 출금은 맞다. 학습용 예제에는 잔액 부족·음수 금액 검사가 없어서 100 입금 후 150 출금하면 -50이다.

필기 실행 주의

Lecture2-1.py, Lecture3-1.py, Lecture3-2.py의 예제들은 대부분 작은따옴표 세 개로 묶여 있다. 이것은 실행 코드가 아닌 문자열 리터럴이다. 예제 한 블록의 감싸는 따옴표만 제거하고 실행하자. 모든 블록을 한꺼번에 해제하면 동일 클래스명 재정의로 추적이 어려워진다.

암기 체크

☐ __age 직접 접근 예외 ☐ getter 반환 ☐ setter 대입 ☐ 잔액 검증 유무

마무리 연습문제

문제 08-1

Student("Hong",20)에서 obj.__age와 obj.getAge()의 차이는?

풀이 메모: _____

문제 08-2

BankAccount 새 객체에서 deposit(70), withdraw(20), withdraw(80)의 반환값은 순서대로?

풀이 메모: _____

문제 08-3

빈칸: def getBalance(self): return ____ (비공개 잔액 속성을 읽어라).

풀이 메모: _____

문제 08-4

직접 작성: Student의 setAge에서 0 이상인 경우에만 나이를 변경하도록 확장하라. 음수이면 기존 값을 유지하라.

풀이 메모: _____

막힐 때만 보는 힌트

- 08-1: 이름 변환과 공용 메소드.
- 08-2: 잔액을 호출마다 갱신한다.
- 08-3: 속성 앞의 self를 잊지 말자.
- 08-4: 조건을 만족할 때만 대입.

PART 09 객체 참조·is·클래스 변수

자료: Lecture 3 p.10-21 / Lecture3-2.py

대입은 복사가 아니다

```
class Television:
    def __init__(self, channel, volume, on):
        self.channel = channel
        self.volume = volume
        self.on = on
t = Television(11, 10, True)
s = t
s.channel = 9
print(t.channel, s is t) # 9 True
```

s=t는 새 객체를 만들지 않는다. 두 이름이 같은 객체를 참조한다. ==는 동등성(정의된 값 비교), is는 객체 동일성이다. 값이 같은 별도 리스트는 ==가 True이고 is는 False이다.

None은 “값 없음”을 나타내는 단일 객체이다. 엄밀히 변수는 None 객체를 참조한다. x is None으로 검사한다.

객체를 함수에 전달

```
def setSilentMode(tv):
    tv.volume = 2
setSilentMode(t)
print(t.volume) # 2
```

매개변수 tv와 t가 같은 객체를 참조하므로 속성 변경이 밖에서도 보인다. 함수 안에서 tv=None으로 이름만 다시 연결하면 바깥 t가 None으로 바뀌지는 않는다. “전달하면 항상 바깥 변수가 바뀐다”는 설명은 틀리다.

인스턴스 변수와 클래스 변수

```
class Television:
    serialNumber = 0
    def __init__(self, channel, volume, on):
        self.channel = channel
        self.volume = volume
        self.on = on
        Television.serialNumber += 1
        self.number = Television.serialNumber
```

serialNumber는 클래스가 공유하는 누적 카운터, number는 각 객체가 생성 당시 받은 번호. 세 객체를 만들면 누적 카운터 3, 각각 number 1/2/3이다. s=t처럼 참조만 추가하면 __init__이 실행되지 않아 카운터도 증가하지 않는다.

필기 serialNumber는 이름이 일관되어 실행은 되지만 자료의 serialNumber와 표기가 다르다. 학습본은 serialNumber로 통일한다. Dog.kind='Bulldog'는 공통 클래스 속성, self.name/self.age는 개별 속성이다. obj.kind='Poodle'을 대입하면 보통 그 객체의 인스턴스 속성이 생겨 클래스 속성을 가린다(보충).

상수 관례

Monster.NORMAL=10 같은 대문자 이름은 상수로 취급하자는 관례이며 변경을 문법적으로 금지하지 않는다. 자료의 VERY STRONG처럼 공백이 들어간 식별자는 불가하므로 VERY_STRONG으로 쓴다. _health의 밑줄 하나는 비공개 의도 관례이며 __health의 이름 변환과 다르다.

암기 체크

☐ 대입/새 생성 구분 ☐ ==/is ☐ 객체 변경/지역 재대입 ☐ 공유 카운터/개별 번호

마무리 연습문제

문제 09-1

a=[1,2]; b=a; c=[1,2]; b.append(3) 후 a, c, a is b, a == c는?

풀이 메모: _____

문제 09-2

def f(t): t=None. myTV를 f(myTV)에 전달하면 myTV도 None이 되는가? 이유는?

풀이 메모: _____

문제 09-3

위 serialNumber 예제에서 a,b 두 객체를 만들고 s=a를 실행했다. 클래스 카운터, a.number, b.number는?

풀이 메모: _____

문제 09-4

class Dog: kind="Bulldog" 이후 a=Dog(); b=Dog(); a.kind="Poodle"을 실행하면 a.kind, b.kind, Dog.kind는?

풀이 메모: _____

막힐 때만 보는 힌트

- 09-1: b=a는 복사가 아니다.
- 09-2: 속성 변경인가 이름 재대입인가?
- 09-3: 생성자 실행 횟수를 센다.
- 09-4: 인스턴스 대입은 클래스 대입과 다르다.

PART 10 특수 메소드·Vector2D·Line

자료: Lecture 4 p.2-9 / Lecture4-1.py

연산자와 메소드 대응

$a+b \rightarrow \text{__add__}$, $a-b \rightarrow \text{__sub__}$, $a*b \rightarrow \text{__mul__}$, $a/b \rightarrow \text{__truediv__}$, $a//b \rightarrow \text{__floordiv__}$, $a\%b \rightarrow \text{__mod__}$, $a**b \rightarrow \text{__pow__}$. 비교 $< \rightarrow \text{__lt__}$, $\leq \rightarrow \text{__le__}$, $> \rightarrow \text{__gt__}$, $\geq \rightarrow \text{__ge__}$, $= \rightarrow \text{__eq__}$, $!= \rightarrow \text{__ne__}$. $\text{str}(a)$, $\text{print}(a)$ 의 문자열 표현은 __str__ 으로 지정할 수 있다. 기본 학습 상황에서는 왼쪽 객체가 self , 오른쪽 객체가 other 이다(반사 연산 등은 범위 밖).

Vector2D 전체 예제

```
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __add__(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)
    def __sub__(self, other):
        return Vector2D(self.x - other.x, self.y - other.y)
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y
    def __str__(self):
        return '(%g, %g)' % (self.x, self.y)
u = Vector2D(0, 1)
v = Vector2D(1, 0)
w = Vector2D(1, 1)
a = u + v
print(a, a == w, a is w) # (1, 1) True False
```

덧셈은 원본 u/v 를 바꾸지 않고 새 벡터를 반환한다. $==$ 는 x 와 y 가 모두 같아야 참이다. 동일한 좌표여도 별도 객체면 is 는 False .

Line은 반환 자료형이 다르다

```
class Line:
    def __init__(self, length):
        self.length = length
    def __add__(self, other):
        return self.length + other.length
    def __lt__(self, other):
        return self.length < other.length
    def __eq__(self, other):
        return self.length == other.length
```

$\text{Line}(10)+\text{Line}(5)$ 는 정수 15. Vector2D 의 $+$ 는 새 Vector2D . 연산자라고 해서 반드시 같은 클래스 객체를 반환하는 것은 아니다. 자료 p.8의 __eq__ 설명에 “짧으면 True ”는 잘못된 설명이며 실제 $==$ 코드처럼 길이가 같으면 True 이다.

필기 수정: 짧은 선 삭제

원본 필기는 line1=Line(10), line2=Line(5)에서 else로 가서 del(line1)을 실행한다. 실제로는 긴 선을 지운다. 과제의 “짧은 선 삭제” 의도대로라면:

```
if line1 < line2:
    print('선2가 더 길니다.')
    del line1
elif line1 == line2:
    print('두 선의 길이가 같습니다.')
else:
    print('선1이 더 길니다.')
    del line2
```

__del__과 del 주의

필기 __del__은 제거 메시지를 출력한다. del line1은 이름/참조를 제거하며 객체 자체의 즉시 파괴를 항상 보장하지 않는다. 다른 참조가 남아 있으면 객체는 살아 있다. __del__ 호출 시점·프로그램 종료 시 출력 순서는 구현과 환경에 영향을 받으므로 시험 추적에서 명시 실행과 종료 정리를 구별한다.

암기 체크

☐ self/other ☐ __str__은 문자열 반환 ☐ Vector/Line 덧셈 반환형 ☐ ==/is ☐ del 참조

마무리 연습문제

문제 10-1

위 벡터 클래스에서 u=Vector2D(2,3), v=Vector2D(1,5). print(u+v), print(u-v), print(u==v)의 결과를 쓰라.

풀이 메모: _____

문제 10-2

Line(10)+Line(5)와 Vector2D(0,1)+Vector2D(1,0)의 반환 자료형을 쓰라.

풀이 메모: _____

문제 10-3

원본 필기의 Line(10), Line(5) 비교에서 어느 else가 실행되며 어떤 이름이 삭제되는가? 의도대로 수정하라.

풀이 메모: _____

문제 10-4

직접 작성: radius를 초기화하고 같은 반지름인지 ==로 비교하는 Circle 클래스를 작성하라. 별도 생성한 Circle(10) 두 개의 ==와 is도 예상하라.

풀이 메모: _____

막힐 때만 보는 힌트

- 10-1: x, y 를 각각 계산한다.
- 10-2: 각 `__add__`의 return을 읽자.
- 10-3: $10 < 5$, $10 == 5$ 를 검사하자.
- 10-4: `__eq__`에서 반지름을 비교한다.

시험 직전 최종 점검

- input은 str이다. □ =와 ==를 구별한다. □ /와 //는 다르다.
- range 끝값과 슬라이싱 끝값은 제외한다. □ break 종료 / continue 다음 반복.
- print와 return은 다르다. □ global 없이 지역 대입은 전역을 바꾸지 않는다.
- 2차원 리스트는 [행][열]. □ 딕셔너리는 중괄호, get의 기본 반환은 None.
- set은 순서 없음, {}는 dict. □ find 실패 -1 / index 실패 예외.
- 한 원소 튜플은 쉼표. □ self는 호출 대상 객체. □ __init__은 초기화.
- __이름은 이름 변환. □ 대입은 객체 복사 아님. □ == 동등성 / is 동일성.
- 클래스 카운터와 객체 번호는 별개. □ 특수 메소드 반환값을 확인한다.
- Line 필기의 긴 선 삭제를 의도와 구별한다. □ del과 즉시 소멸은 같지 않다.

추천 복습 순서

첫 회독: 01-06 기초 문법. 둘째 회독: 07-10 객체 흐름을 손으로 추적. 마지막: 40문제 중 틀린 문제만 다시 풀고 위 체크리스트를 말로 설명한다.